TECHNOVA SOLUTIONS PVT. LTD.

TESTING STANDARDS

Document ID	TN-ENG-STD-006
Document Type	Engineering Testing Standards
Document Owner	Engineering / Quality Engineering
Approving Authority	VP Engineering / CTO
Classification	Internal — Engineering
Version	1.0
Effective Date	01 April 2026
Review Cycle	Annual
Status	Active
Supersedes	New Standard

1. Purpose

These Testing Standards define the minimum expectations for planning, designing, implementing, executing, and maintaining software tests at TechNova Solutions Pvt. Ltd. (“TechNova” or “the Company”).

The standards are intended to provide appropriate confidence that software changes satisfy requirements, handle important failure conditions, protect data, and can be operated reliably.

2. Scope

These standards apply to applications, services, APIs, libraries, scripts, automation, infrastructure code, databases, data-processing systems, AI/ML systems, and other engineering software.

The required depth of testing should be proportional to business impact, technical complexity, security sensitivity, and operational risk.

3. Testing Principles

- Test behavior and risk, not merely code coverage numbers.
- Automate repeatable tests wherever practical.
- Test early and continuously throughout development.
- Keep tests reliable and maintainable.
- Prefer deterministic tests.
- Test important failure and boundary conditions.
- Use production-like validation for important integration behavior.
- Do not treat a green test suite as proof that every requirement is correct.

4. Testing Responsibilities

Role	Typical Responsibilities
Developer	Create and maintain appropriate unit and integration tests and validate changes locally.
Reviewer	Assess whether testing is appropriate for the change and risk.

Role	Typical Responsibilities
QA / Quality Engineer	Design broader validation strategies, test scenarios, and quality assessments where applicable.
Technical Lead	Ensure appropriate testing depth for architecture and system risk.
Service Owner	Ensure production systems have suitable validation, monitoring, and operational tests.

5. Test Planning

Testing should begin with understanding what must be correct and what could fail.

- Identify acceptance criteria.
- Identify important user and business workflows.
- Identify security and authorization requirements.
- Identify data-integrity risks.
- Identify external dependencies.
- Identify important edge cases and failure modes.
- Determine the appropriate testing levels.

6. Test Levels

Test Level	Primary Purpose
Unit Testing	Validate isolated functions, classes, modules, or small units of behavior.
Integration Testing	Validate interactions between components or external dependencies.
System Testing	Validate the behavior of the integrated application or service.
End-to-End Testing	Validate important complete workflows across system boundaries.
Acceptance Testing	Confirm that the implementation satisfies defined business or product expectations.
Performance Testing	Assess response time, throughput, resource use, or scalability where relevant.
Security Testing	Identify vulnerabilities and validate security controls.

7. Unit Testing

Unit tests should validate important logic in isolation from unrelated external systems where practical.

- Test normal behavior.
- Test boundary conditions.
- Test meaningful invalid inputs.
- Test important error handling.
- Keep tests focused on one logical behavior.
- Keep unit tests fast enough to run frequently.

8. Unit Test Design

- Use clear test names.
- Keep setup understandable.
- Avoid excessive mocking.
- Mock only dependencies that should be isolated or are expensive/unreliable.

- Do not make tests depend unnecessarily on execution order.

9. Integration Testing

Integration tests validate interactions that unit tests cannot adequately represent.

- Database interactions
- API calls
- Message queues
- File or object storage
- Authentication and authorization integrations
- External service contracts
- Component-to-component behavior

10. System Testing

System tests should validate the integrated behavior of a service or application against meaningful requirements.

They should focus on important workflows rather than duplicating every unit-level test.

11. End-to-End Testing

End-to-end tests should cover a selected set of critical user or business workflows.

- Authentication and authorization workflows
- Critical transaction flows
- Important document or data workflows
- High-value user journeys
- Critical integrations

12. Acceptance Testing

Acceptance tests should be derived from agreed requirements and acceptance criteria.

A change should not be considered complete solely because technical tests pass if the implemented behavior does not meet the intended business requirement.

13. Regression Testing

Regression testing verifies that existing behavior has not been unintentionally broken by a change.

- Add regression tests for significant defects.
- Run relevant existing suites after material changes.
- Prioritize critical workflows.
- Maintain regression suites as the product evolves.

14. Test Case Design

- Normal valid input
- Boundary values
- Empty or missing values
- Invalid input

- Unexpected input formats
- Permission differences
- Dependency failures
- Timeouts
- Duplicate requests
- Large or unusual data volumes where relevant

15. Boundary Testing

Boundary testing should be used where behavior changes at limits.

Examples include maximum field length, minimum and maximum quantities, date ranges, file sizes, pagination boundaries, rate limits, and access thresholds.

16. Negative Testing

Negative tests verify that invalid or unauthorized operations fail safely.

- Malformed requests
- Missing required fields
- Invalid credentials
- Insufficient permissions
- Unavailable dependencies
- Invalid state transitions
- Unsupported values

17. Authorization Testing

Systems with role-based access must test both permitted and denied operations.

- Employee access
- HR access
- Manager access
- Administrator access
- Department restrictions
- Document-level permissions
- Unauthorized retrieval attempts

18. Enterprise Knowledge and RAG Testing

For TechNova's enterprise knowledge assistant, testing must verify that authorization is enforced before protected content reaches downstream generation.

- Authorized users can retrieve permitted documents.
- Unauthorized users cannot retrieve restricted documents.
- Department restrictions are respected.
- Restricted chunks do not enter LLM context.
- Citations do not expose unauthorized sources.

- Permission changes are reflected in retrieval behavior.
- Document versions do not accidentally bypass access restrictions.

19. API Testing

- Validate request and response formats.
- Test authentication.
- Test authorization.
- Test error responses.
- Test invalid input.
- Test compatibility requirements.
- Test important rate and timeout behavior where applicable.

20. Database Testing

- Validate schema behavior.
- Test important constraints.
- Test transaction behavior.
- Test migration behavior.
- Test data integrity.
- Test representative query behavior.
- Verify authorization around sensitive data.

21. Data Validation

Data-processing systems should test transformations, validation rules, missing values, duplicate data, unexpected formats, and important volume or boundary conditions.

Test datasets should avoid unnecessary use of real sensitive information.

22. File and Document Processing

- Valid document formats
- Unsupported formats
- Corrupt or incomplete files
- Empty documents
- Large documents
- Scanned documents requiring OCR
- Unexpected encoding
- Tables and structured content
- Metadata extraction

23. AI and Machine Learning Testing

AI/ML systems require testing beyond traditional software correctness.

- Input validation

- Model output behavior
- Relevant accuracy or quality measures
- Robustness to unexpected inputs
- Access-control behavior
- Data leakage risks
- Model and dependency compatibility
- Latency and resource behavior
- Failure and fallback behavior

24. RAG Evaluation

- Retrieval relevance
- Recall of appropriate source content
- Reranking quality where used
- Context completeness
- Groundedness of generated answers
- Citation correctness
- Permission filtering
- Behavior when no authorized source exists

25. Prompt and LLM Testing

Applications using large language models should test representative prompts and failure scenarios.

- Normal user questions
- Ambiguous questions
- Questions with no matching knowledge
- Attempts to request restricted information
- Prompt injection or instruction-conflict scenarios
- Long-context behavior
- Incorrect or unsupported assumptions

26. Security Testing

- Authentication controls
- Authorization controls
- Input validation
- Injection risks
- Secret exposure
- Session and token behavior
- Access to protected resources
- Security-sensitive configuration

27. Performance Testing

Performance testing should be performed when response time, throughput, scale, or resource consumption materially affects the system.

- Define realistic workloads.
- Use representative data volumes.
- Measure response time and throughput.
- Observe CPU, memory, database, network, and other relevant resources.
- Identify bottlenecks and thresholds.

28. Load Testing

Load testing evaluates system behavior under expected or planned concurrent usage.

Test loads should be based on realistic operational expectations rather than arbitrary numbers.

29. Stress Testing

Stress testing evaluates behavior beyond expected load or resource limits.

- Identify failure points.
- Observe degradation behavior.
- Verify recovery after load is reduced.
- Ensure failures do not corrupt important data.

30. Reliability Testing

- Dependency failures
- Service restarts
- Network interruptions
- Timeouts
- Retry behavior
- Partial failures
- Recovery procedures

31. Test Environments

Test environments should be sufficiently representative of the target environment for the behavior being validated.

- Document meaningful environment differences.
- Use controlled test data.
- Protect test credentials.
- Keep test configuration reproducible where practical.

32. Test Data

Test data should be designed to represent important real-world conditions without unnecessarily exposing sensitive information.

- Use synthetic or anonymized data where practical.
- Include normal and edge-case datasets.
- Control access to restricted test data.

- Do not copy production data into test environments without authorization and appropriate safeguards.

33. Test Isolation

Tests should avoid unnecessary dependence on shared mutable state.

- Reset test state where required.
- Use isolated databases or schemas where practical.
- Control external service dependencies.
- Avoid tests that pass or fail based on another test's execution.

34. Deterministic Tests

Automated tests should produce consistent results for the same conditions.

When randomness, time, concurrency, or external services make determinism difficult, the test should control or appropriately account for those factors.

35. Flaky Tests

A flaky test is a test that intermittently passes or fails without a relevant product change.

- Investigate flaky tests promptly.
- Identify root causes.
- Do not permanently ignore failures by hiding or deleting the test.
- Temporarily quarantine a test only through an approved process with an owner and follow-up.

36. Test Automation

- Automate stable, repeatable regression checks.
- Keep automated tests maintainable.
- Run fast tests frequently.
- Run broader suites at appropriate pipeline stages.
- Keep test results visible to the engineering team.

37. Continuous Integration

CI pipelines should execute the required automated test suites for the affected code.

- Unit tests
- Integration tests where configured
- Static analysis
- Security checks where required
- Build validation

38. Continuous Delivery and Deployment Validation

Deployment pipelines should perform appropriate pre-production and post-deployment checks.

- Artifact validation
- Environment health checks
- Smoke tests

- Critical workflow verification
- Rollback or recovery validation where applicable

39. Smoke Testing

Smoke tests provide a rapid check that a newly deployed system is functioning at a basic level.

- Service starts successfully.
- Critical dependencies are reachable.
- Authentication works where applicable.
- Important endpoints or workflows respond.
- Basic health indicators are normal.

40. User Acceptance and Business Validation

Where business behavior is important, appropriate stakeholders should validate that the implementation meets the intended outcome.

Technical test success does not replace required business acceptance.

41. Test Coverage

Coverage metrics can identify untested areas but should not be treated as the sole measure of quality.

- Prioritize important behavior.
- Cover high-risk branches and failure paths.
- Use coverage trends as a signal.
- Avoid writing meaningless tests solely to increase a percentage.

42. Code Coverage Thresholds

Projects may establish coverage thresholds based on technology and risk. A threshold should encourage useful testing rather than discourage necessary refactoring or meaningful test design.

43. Test Review

Tests are code and should receive appropriate review.

- Check whether the test actually verifies the intended behavior.
- Check test clarity.
- Check important edge cases.
- Check for brittle assumptions.
- Check that mocks and fixtures do not hide real defects.

44. Mocking and Stubbing

Mocks and stubs should isolate dependencies when appropriate without creating unrealistic behavior.

- Do not mock the system under test unnecessarily.
- Keep mock behavior aligned with real contracts.
- Use integration tests to validate important real interactions.

45. Contract Testing

Where independent services communicate through APIs or events, contract testing may be used to detect incompatible changes.

Contracts should represent meaningful consumer expectations.

46. Compatibility Testing

- API versions
- Database schema compatibility
- Client compatibility
- Browser or platform requirements where applicable
- Message or event schema compatibility

47. Accessibility and Usability Testing

Where software has user interfaces, appropriate accessibility and usability validation should be included according to product requirements and supported platforms.

48. Localization and Internationalization Testing

Products supporting multiple languages, locales, currencies, dates, or time zones should test representative locale-specific behavior.

49. Test Security and Confidentiality

Test results, screenshots, logs, datasets, and defect reports must not unnecessarily expose credentials, personal data, customer information, source code, or other restricted content.

50. Defect Reporting

- Describe the observed behavior.
- Provide reproducible steps where possible.
- Identify expected versus actual behavior.
- Include environment and version information.
- Attach relevant logs or evidence without exposing unnecessary sensitive information.
- Identify severity and impact appropriately.

51. Defect Severity

Severity	Typical Meaning
Critical	Systemic failure, severe security issue, major data-integrity problem, or critical business impact.
High	Major functionality failure or significant operational/customer impact.
Medium	Meaningful defect with a workaround or limited impact.
Low	Minor functional, cosmetic, or low-impact issue.

52. Defect Retesting

After a defect is fixed, the responsible tester or engineer should verify the correction and perform appropriate regression testing to ensure related behavior remains correct.

53. Release Readiness

Before production release, the responsible team should evaluate whether required tests have passed and whether known defects and risks are acceptable.

- Required automated checks passed.
- Critical workflows validated.
- Known high-risk defects addressed or explicitly accepted.
- Security requirements satisfied.
- Deployment and rollback considerations addressed.

54. Test Failure and Escalation

A failed test should be investigated rather than automatically ignored.

- Determine whether the failure represents a product defect, test defect, environment issue, or infrastructure issue.
- Record the appropriate action.
- Escalate significant failures.
- Do not bypass a required quality gate without authorization.

55. Testing in Production

Production testing must be controlled and should not create unnecessary customer, data, availability, or security risk.

- Use approved smoke or health checks.
- Use synthetic transactions where appropriate.
- Do not expose real customer data unnecessarily.
- Do not perform destructive tests without explicit authorization.

56. Chaos and Resilience Testing

Critical systems may use controlled resilience or fault-injection testing to validate recovery behavior.

Such tests require appropriate authorization, safeguards, monitoring, and rollback or termination procedures.

57. Test Documentation

Important test strategies, environments, assumptions, datasets, acceptance criteria, and known limitations should be documented where appropriate.

58. Test Evidence

Test results may be retained as evidence for release decisions, defect investigation, audits, or operational learning.

Evidence should be retained and protected according to applicable Company requirements.

59. Third-Party Services

Where systems depend on external services, tests should cover important contract behavior, error responses, timeouts, rate limits, and dependency failures where practical.

60. Testing Changes to Security and Access Controls

- Test authorized access.
- Test unauthorized access.
- Test role changes.
- Test department restrictions.
- Test revoked access.
- Test document-level permissions.
- Test privileged operations.

61. Testing Data Retention and Deletion

Systems with retention or deletion requirements should test that records are retained for the intended period and deleted or archived according to approved rules.

62. Testing Backups and Recovery

Critical systems should validate backup and recovery procedures at an appropriate frequency.

- Verify backups complete successfully.
- Verify backups are usable.
- Test restoration where required.
- Measure recovery behavior against defined objectives.
- Document significant recovery-test findings.

63. Testing Observability

Tests should verify that important failures produce appropriate logs, metrics, alerts, or other operational signals where observability is part of the system requirement.

64. Testing Configuration

Material configuration changes should be tested for correct behavior, environment compatibility, security, and failure handling.

65. Testing Infrastructure

Infrastructure changes should be validated for provisioning, permissions, networking, availability, monitoring, and recovery behavior as appropriate.

66. Testing Generated or AI-Assisted Code

AI-generated or automatically generated code must receive the same appropriate testing as manually written code.

- Verify functional behavior.
- Test edge cases.
- Check security-sensitive behavior.
- Review generated assumptions.
- Do not rely on generation-tool confidence or explanations as evidence of correctness.

67. Testing Model and Retrieval Updates

Changes to AI models, prompts, embeddings, retrieval algorithms, rerankers, chunking strategies, or knowledge sources should be evaluated against representative evaluation cases before production use.

- Compare relevant quality metrics.
- Check retrieval relevance.
- Check answer groundedness.
- Check citations.
- Check access-control behavior.
- Check latency and resource impact.

68. Test Maintenance

- Remove obsolete tests when requirements are intentionally removed.
- Update tests when behavior changes.
- Refactor repetitive test code.
- Keep fixtures and test data understandable.
- Review flaky or slow tests periodically.

69. Exceptions

A project may request an exception where a testing requirement is technically inappropriate or disproportionate to risk.

Exceptions should identify the requirement, reason, risk, compensating controls, owner, duration, and approval.

70. Testing Anti-Patterns

- Testing only the happy path.
- Writing tests solely to increase coverage.
- Ignoring flaky tests indefinitely.
- Using production data without authorization.
- Disabling tests because they fail without investigating the cause.
- Relying entirely on end-to-end tests.
- Relying entirely on unit tests for integration behavior.
- Testing security only after release.
- Assuming AI-generated code is correct without validation.

71. Frequently Asked Questions

Does every function need a unit test?

Not necessarily. Testing depth should reflect behavior, risk, complexity, and the value of the test.

Is 100% code coverage required?

Coverage requirements depend on the project. Coverage is a useful signal but is not the sole measure of test quality.

What should I do with a flaky test?

Investigate it, identify the cause, and fix or appropriately quarantine it through an approved process rather than ignoring it permanently.

Can I use production data for testing?

Only when authorized and appropriately protected. Synthetic or anonymized data should be preferred where practical.

Do access-control changes need special testing?

Yes. Both allowed and denied scenarios should be tested, including role, department, document, and privileged-access behavior where applicable.

How should RAG systems be tested?

Evaluate retrieval relevance, context, groundedness, citations, no-answer behavior, and authorization filtering.

Can AI-generated code skip testing?

No. Generated code is subject to the same appropriate testing requirements.

What if a required test fails because the environment is broken?

Determine the cause, document it, and resolve or appropriately isolate the environment issue. Do not simply treat the test as successful.

Should every deployment run the entire test suite?

The pipeline should run the required suites appropriate to the project and change. Broader suites may run at different stages.

Who decides whether testing is sufficient?

The responsible engineering team, reviewers, technical owners, and other designated stakeholders determine appropriate testing based on requirements and risk.

72. Standard Administration

Engineering owns these Testing Standards and reviews them at least annually or when material changes occur in development practices, architecture, quality requirements, or organizational needs.

The latest approved version maintained in the official Company repository is the authoritative version.

73. Revision History

Version	Effective Date	Description	Owner	Approval
1.0	01 April 2026	Initial release of Testing Standards	Engineering / Quality Engineering	VP Engineering / CTO

74. Related Documents

- Engineering Handbook
- Coding Standards
- Git Workflow
- Code Review Guidelines
- Deployment Policy
- Information Security Policy
- Access Control Policy
- Business Continuity
- Incident Response

END OF DOCUMENT

TN-ENG-STD-006 | Version 1.0 | Internal — Engineering | Engineering / Quality Engineering